

Point It, Strike It: Direction-Conditioned Dynamic Manipulation of Deformable Linear Objects

One model: **Any Rope & Any Point & Any Direction**

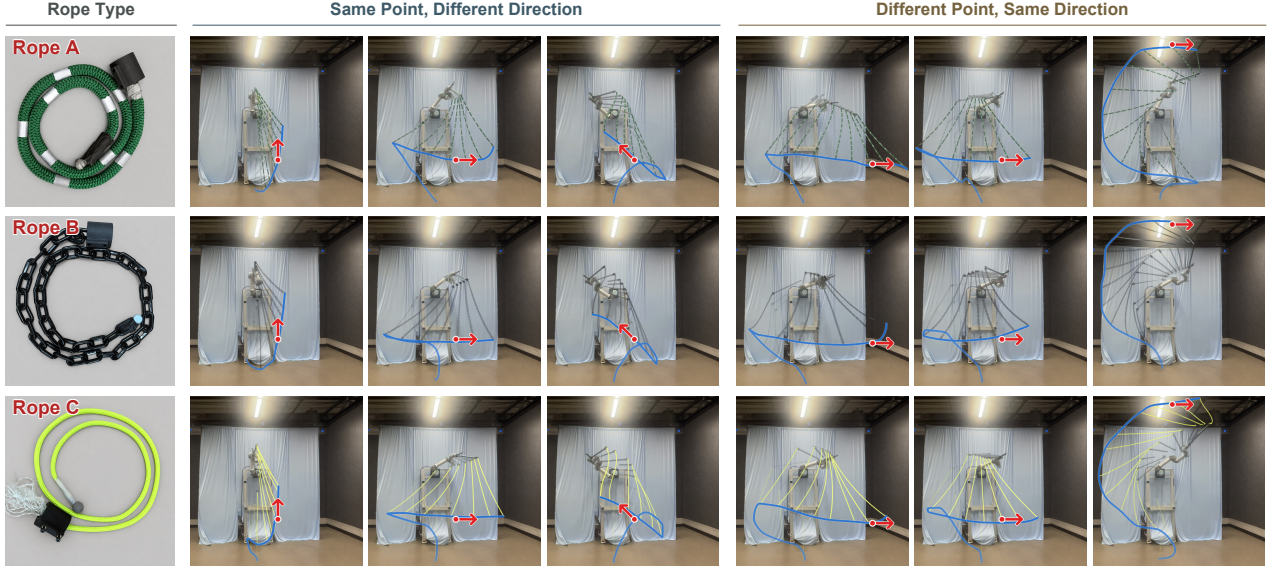


Fig. 1: **One model, any rope, any point, any direction.** Real-world strikes across three DLOs with different physical properties: **Rope A**, a green braided rope; **Rope B**, a black plastic chain; and **Rope C**, a yellow braided cord. For each rope, the robot strikes the same point from different target arrival directions (left) and different points with the same target arrival direction (right). Red dots and arrows indicate the commanded strike position and arrival direction. Each example is executed with a zero-shot strike.

Abstract—Goal-conditioned dynamic manipulation of deformable linear objects has mainly specified goals as positions for a rope tip to reach. Many tasks, however, depend on how the tip arrives. We therefore specify the goal as the tip’s state, its 3D position and arrival direction, reached with one swing anywhere in the workspace and on different ropes. This is challenging because rope dynamics are hard to model, no demonstrations exist, distinct swings reach the same goal with different reliability, and the sim-to-real gap extends beyond the rope. To address these challenges, we extend the state-of-the-art DLO simulator DeformX with GPU acceleration, a stable Cosserat rod solver, and a cross-flow aerodynamic model, yielding DeformX 2.0, which is more than $20,000\times$ faster. We then propose TRACE (Trace-rooted Adaptive Cross-Entropy), which generates striking data by warm-starting each new target from the stored swing whose tip path passes closest to it. Its cost penalizes rope bending and abrupt tip motion to favor repeatable swings. A conditional flow-matching policy trained on this data reaches 92.1% accuracy in simulation. Finally, we propose RECAP (Residual Calibration Policy), which fits the simulator’s rope and rig parameters to a few calibration swings and corrects actions with a delta policy trained in simulation. On a real robot, across three ropes, RECAP raises success within 5 cm from 72% to 87% for position goals, and within 10 cm and 10° from 50% to 79% for goals that also specify the arrival direction.

I. INTRODUCTION

Deformable linear objects (DLOs), such as ropes, cables, wires, and hoses, are widely used in robotic and industrial settings [1], [2]. A line of work studies their *goal-conditioned dynamic manipulation*, where a robot swings or casts the object so that it reaches a commanded goal [2]–[6]. The goals considered so far mainly describe the object’s position, typically a 2D or 3D point for the rope tip to reach. We study a harder goal, where we consider both the 3D position of the rope tip and the direction it should be moving when it arrives. Our 4D goal therefore specifies the *state* of the tip, not only its position. Many tasks depend on how the tip arrives. For example, striking a spinning top in the direction of rotation helps maintain its spin, while striking against it can slow it down. Similarly, knocking an obstruction aside requires a strike that pushes it out of the way.

We study this task on a robot arm that holds one end of a rope. The robot receives a goal $g = (p^*, d^*) \in \mathbb{R}^3 \times S^1$ and must execute a single swing whose tip passes through p^* while moving along d^* . The task covers any point in the reachable workspace, any arrival direction, and any rope. Even for a person, hitting a point with the tip of a rope requires deliberate practice, and reaching it from a specified

direction is harder still. To the best of our knowledge, no prior rope-hitting method addresses a task of this difficulty. For robots, this task is challenging for several reasons:

- **Complex dynamics and object properties:** a swung rope is a stiff, nonlinear, aerodynamically damped continuum whose tip amplifies every command error, and its response depends on properties that vary between ropes.
- **Data sparsity:** no demonstration corpus exists for this task, and it is hard enough for people that teleoperation is impractical.
- **Many-to-one solutions:** very different swings can hit the same target, differing in wind-up, swing plane, or timing. These swings are not equally reliable either. Some hold their tip path under small disturbances, while others change drastically when replayed.
- **Sim-to-real gap:** a swing that works in simulation misses on the real robot, and the mismatch is not confined to the rope. Controller lag, mechanical vibration, and timing error contribute as much as the rope’s own properties.

To address these challenges, we first extend DeformX [2] with GPU acceleration, a stable Cosserat rod solver [7], and a quadratic cross-flow aerodynamic model [8], [9], yielding an accurate and efficient DLO simulator we call DeformX 2.0. We then propose **TRACE** (Trace-rooted Adaptive Cross-Entropy) to generate striking data entirely in this simulator. Starting from one swing verified on hardware, TRACE warm-starts the cross-entropy method for each new target from the stored swing whose tip path passes closest to it. Its cost favors an extended rope and smooth tip motion, so the swings it keeps stay repeatable. A conditional flow-matching policy [10] then learns to strike from this data alone. The network learns to generate suitable swing patterns for different goals, with goal conditioning in every network block to improve striking accuracy [11], [12]. We call the swing it generates for a goal the *base action*. Finally, we propose **RECAP** (Residual Calibration Policy) to adapt the policy to a real rope. RECAP first searches for the simulator’s rope and rig parameters that best reproduce a few calibration swings. Conditioned on these parameters, a delta policy predicts several corrections to the base action. The fitted simulator then replays the base and corrected actions, and the robot executes the best one. Because the delta policy is trained entirely in simulation, the real robot is needed only for the calibration swings. A new rope is calibrated once, and each goal afterwards takes a zero-shot strike. Our main contributions are:

- 1) **DeformX 2.0**, which extends DeformX [2] with GPU acceleration, a stable Cosserat rod solver, and a quadratic cross-flow aerodynamic model. It simulates ropes more than $20,000\times$ faster than DeformX.
- 2) **TRACE**, a data generation method that produces smooth striking data entirely in simulation. In simulation, TRACE covers 98.3% of the workspace, against 49.5% for CEM with an example swing, and a policy trained on its data reaches 92.1% success after candidate selection.
- 3) **4D goal-conditioned flow-matching policy**, to our

knowledge the first rope-striking policy conditioned on 4D goal. It injects the goal into every residual block through zero-initialized modulation, which keeps the distinct swings that reach a goal apart instead of mixing them. Learned from simulation data alone, it reaches 92.1% accuracy within 5 cm and 10° .

- 4) **RECAP**, a residual calibration policy that adapts the base action to a real rope. It fits the simulator’s rope and rig parameters to a few calibration swings. Conditioned on these parameters, a delta policy trained entirely in simulation predicts corrections to the base action, and the fitted simulator selects the best candidate. On hardware, across three ropes, it raises success within 5 cm from 72% to 87% for position goals and from 28% to 52% for position-and-direction goals.

II. RELATED WORK

A. Goal-conditioned dynamic rope manipulation

Robots have learned to swing ropes and cables toward commanded goals. IRP [3] hits 2D targets with a robot-held rope by correcting the action over repeated real-world attempts. Whip primitives [5] reach 3D points with low-dimensional open-loop motions, and Lost Arc [4] vaults, knocks, and weaves a cable around obstacles. Wiggle&Go [6] strikes 3D points on new ropes. It predicts the rope’s simulation parameters from a short wiggle with a network trained in simulation, then optimizes the motion for each goal in the simulator. DeformX [2] reaches 2D targets in a fixed plane with per-step reinforcement learning. These goals mainly describe where the tip should be and leave how it arrives unconstrained. Several of these systems are also restricted to a plane or need repeated real attempts for each goal. We specify the goal by both the 3D position of the tip and its arrival direction, and reach any such goal in the workspace with a single zero-shot strike.

B. Planner-generated training data

When demonstrations are scarce, planners can generate training data instead. Zhu et al. [13] show that sampling-based planners produce inconsistent demonstrations on which even expressive diffusion policies degrade. PhysicGen [14] obtains consistent data by re-solving trajectory optimization warm-started from a few demonstrations under perturbed conditions. Dynamic rope striking adds two difficulties. There is no demonstration set to warm-start from, and swings that reach the same goal differ in how reliably they transfer to hardware. TRACE starts from a single swing verified on hardware and warm-starts each new target from the stored swing whose tip path passes closest to it, which keeps the solutions for nearby goals consistent. Its cost favors an extended rope and smooth tip motion to keep swings repeatable, and each verified swing is labeled with its achieved outcome as in hindsight relabeling [15], [16].

C. Generative policies for multimodal action distributions

Diffusion Policy [17] and flow matching [10], [18] model distributions over actions instead of a single prediction. DAMP [19] builds on motion manifold primitives [20] and

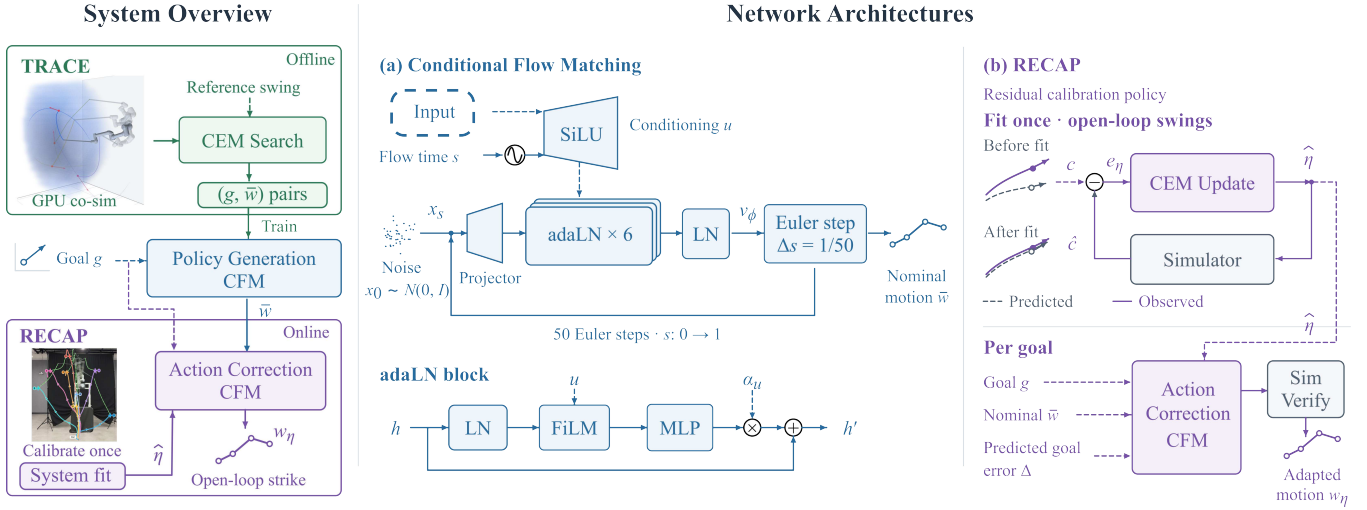


Fig. 2: **System overview and network architectures.** Left: TRACE generates goal–action pairs in GPU co-simulation to train the nominal policy. RECAP calibrates the real system once and corrects the action for each goal before a single open-loop strike. (a) Conditional flow matching (CFM) is designed to generate nominal actions conditioned on g and corrected actions conditioned on $(g, \bar{w}, \Delta, \hat{\eta})$. (b) RECAP iteratively fits rope and rig parameters to measured calibration swings. These estimates condition the correction network to generate actions adapted to the real system. Dashed lines indicate input.

applies conditional flow matching to goal-conditioned throwing. In rope striking, distinct motion patterns reach the same goal, making the action distribution inherently multimodal. A regressor tends to average across these patterns, often producing an invalid swing. A generative model avoids this average, but its samples still spread between patterns when the goal enters only at the network input. Our flow-matching policy injects the goal into every residual block through zero-initialized modulation [11], [12], which keeps its samples within a pattern and raises one-shot accuracy.

D. Sim-to-real calibration

One line of work identifies physical parameters and uses them downstream. Wiggle&Go [6] predicts rope parameters with a learned network and optimizes each goal in simulation with them, and GenDOM [21] conditions a policy on parameters estimated from one real demonstration. Another line corrects actions from observed outcomes. TossingBot [22] learns a residual on a ballistic release velocity, and IRP [3] refines the swing action over repeated real attempts at each target. Our proposed RECAP fits rope and rig parameters to a few calibration swings for each rope, and a delta policy trained in simulation and conditioned on these parameters corrects the base action for the error the fit still misses.

III. PROBLEM FORMULATION

A robot holds one end of a rope and executes a single swing. Given a target position and arrival direction, the goal is to generate a robot trajectory whose rope tip reaches the target while moving in the desired direction.

The input is a goal:

$$g = (p^*, d^*) \in \mathbb{R}^3 \times S^1, \quad d^* = (\cos \theta^*, \sin \theta^*). \quad (1)$$

where p^* is the desired tip position, d^* is the desired arrival direction, and θ^* is the desired arrival angle. The output is a parameter vector $w \in \mathbb{R}^m$ that decodes into an executable joint trajectory $q_w(t)$ over a fixed time horizon. The action w specifies joint-space waypoints and a duration. A clamped cubic spline through the waypoints reconstructs $q_w(t)$, and the duration is adjusted so that the trajectory satisfies the joint-velocity limit. The rope’s response to a robot action depends on the properties of the rope and robot setup, which we denote collectively by η .

A goal does not uniquely determine a swing. Distinct robot trajectories may bring the rope tip to the same goal. We therefore represent the possible swings with a conditional distribution $p_\eta(w | g)$ over feasible trajectories.

IV. METHODS

A. Overview

We decompose the problem into nominal inverse learning and rope-specific adaptation. First, we use a simulator with nominal rope properties $\bar{\eta}$ to generate feasible goal–trajectory pairs. A conditional flow-matching policy [10] then learns the nominal inverse distribution

$$q_\phi(w | g) \approx p_{\bar{\eta}}(w | g). \quad (2)$$

This stage captures the general relationship between desired rope-tip outcomes and robot trajectories.

Second, the sim-to-real stage uses a few calibration swings to characterize the real rope system η . Action adaptation T_ψ then corrects a nominal action $\bar{w}$ with the goal g and the calibrated system:

$$w_\eta = T_\psi(\bar{w}, g, \eta). \quad (3)$$

Calibration is performed once per system, after which each goal is executed with an open-loop swing.

Algorithm 1 TRACE data generation

Require: reference w_{ref} ; targets $\mathcal{G}$; passes P

```
1: Initialize stored swings and tip paths  $\mathcal{S}$  from  $(g_{\text{ref}}, w_{\text{ref}})$ 
2: for  $P$  passes do
3:   for all unsolved  $g \in \mathcal{G}$ , nearest reference first do
4:     Choose  $w_0$  from  $\mathcal{S}$  with the closest tip path to  $g$ 
5:      $w^* \leftarrow \text{CEM}(w_0; g, J)$ 
6:     If  $M(w^*) \leq 1$ , add  $(g, w^*)$  to  $\mathcal{S}$ 
7:   end for
8: end for
9: Refine solved targets from neighboring actions in
   spanning-tree order; replace only if  $M \leq 1$ .
10: return  $\mathcal{S}$  hindsight-labeled on  $M \leq 1$  segments
```

B. Learning robust motion patterns

Dynamic rope manipulation is sensitive to small changes in robot motion, yet distinct swings can reach the same goal. These swings may differ in their wind-up, swing plane, or timing. We refer to groups of similar swings as *motion patterns*. Within some patterns, changes in robot motion produce gradual changes in the rope trajectory. Interpolating between different patterns, however, can produce unsuccessful swings.

This multiplicity motivates a generative policy rather than deterministic regression. A regressor trained with squared error tends toward the conditional mean, which may average incompatible swings. Conditional flow matching [10] instead models a distribution of actions for each goal, allowing it to represent distinct motion patterns.

1) **Data generation:** We call our generator **TRACE** (Trace-rooted Adaptive Cross-Entropy). It uses the cross-entropy method (CEM) [23] to generate accurate strikes while favoring robust rope motion. We guide CEM with:

$$J(w) = L_{\text{target}} + L_{\text{spd}} + L_{\text{lim}} + L_{\text{curv}}. \quad (4)$$

The target, speed, and limit terms penalize position and direction errors, insufficient tip speed, and constraint violations. We accept a swing when these normalized losses satisfy $M(w) = \max\{L_{\text{target}}, L_{\text{spd}}, L_{\text{lim}}\} \leq 1$. The curvature term L_{curv} favors an extended rope and smooth tip motion, encouraging repeatable swings that tolerate disturbances and sim-to-real mismatch.

Our proposed TRACE keeps the solutions for nearby goals consistent by choosing where each search starts. We initialize the search with a manually tuned swing, which has verified robustness on hardware, providing guidance for motion properties not fully captured by the objective. Each new target is warm-started from the stored swing whose tip path passes closest to it, rather than the swing that solves the nearest target. This encourages CEM to extend an existing motion pattern across nearby goals. A final refinement pass re-solves targets from neighboring actions and keeps only verified solutions. Additional goal-action pairs are then sampled around each successful pass.

2) **Flow matching model:** Conditional flow matching (CFM) learns to transform Gaussian noise into swing param-

eters [10], [18]. We normalize each action w dimension using its training-set mean and standard deviation to get normalized action x . Given noise $x_0 \sim \mathcal{N}(0, I)$, a normalized training action x_1 paired with g , and flow time $s \sim \mathcal{U}[0, 1]$, we form $x_s = (1 - s)x_0 + sx_1$ and minimize

$$\mathcal{L}(\phi) = \mathbb{E} \left[\|v_\phi(x_s, s, g) - (x_1 - x_0)\|^2 \right]. \quad (5)$$

The velocity network receives goal and flow-time conditioning in every residual block through adaLN-Zero [11]. The goal’s direction component is encoded as $\sin(\theta)$ and $\cos(\theta)$ to avoid discontinuities at angle wraparound; flow time uses a sinusoidal encoding. These encodings form a vector u , which controls feature scaling, shifting, and a gated residual update:

$$h' \leftarrow h + \alpha_u \odot f(\text{FiLM}_u(\text{LN}(h))), \quad (6)$$

where h is the hidden state, f is an MLP, and α_u is a conditioning-dependent gate. FiLM applies feature-wise scaling and shifting [12]. The modulation outputs, gates, and final output head are zero-initialized. We use 6 adaLN blocks of width 512 (11.3M parameters), as illustrated in Fig. 2.

At inference, sampling provides small variations in the predicted swing that can improve target accuracy through simulated selection. For each goal, we integrate the learned velocity field from 64 independent noise samples using 50 Euler steps, simulate these candidates, and select the action w with the lowest J loss. Policy generation and action correction use the same conditional flow-matching architecture, with input dimensions adapted to their conditioning variables: g for policy generation and $(g, \bar{w}, \Delta, \hat{\eta})$ for action correction.

C. RECAP: residual calibration policy

The flow-matching policy of Sec. IV-B.2 is trained on data from the nominal simulator $\bar{\eta}$. Directly running action $\bar{w}$ from the policy generation CFM would likely fail on a real robot due to sim-to-real gaps. We designed RECAP to identify and compensate for these gaps by executing a few calibration swings for each new rope, fitting parameters to this observation c , and using an action correction CFM to produce corrected action w_η .

System identification. The robot executes 8 fixed calibration swings, while motion capture records the rope-tip trajectories c . Through experiments, we identified 11 parameters η that we can tune to approximate the real setup, which helps us build an approximate simulator. We then use

Engine	Envs	Update rate (Hz)	Step time (ms)	Sim-to-wall time ratio
DeformX [2] (CPU, 1 thread)	1	12.9	77.5	0.22
Ours (RTX 4090)	1	136	7.4	2.3
Ours (RTX 4090)	1024	104	9.6	1780
Ours (RTX 4090)	2048	77.3	12.9	2640
Ours (RTX 4090)	8192	34.5	29.0	4710

TABLE I: Simulation throughput on the same 1 m, 20-segment rope, on one RTX 4090. Rates are per control step of the whole batch.

Rope	Method	3D goal (position)					4D goal (position and direction)					
		SR@5 (%)	SR@10 (%)	Distance (cm)	Best (cm)	Worst (cm)	SR@5 (%)	SR@10 (%)	Distance (cm)	Best (cm)	Worst (cm)	Angle (°)
Rope A	Base	68	91	4.8±4.9	1.3±1.1	15.4±5.9	23	41	13.4±12.0	1.4±0.5	38.7±25.4	9.8±17.3
	Baseline	65	96	4.4±2.8	1.3±0.4	11.9±2.5	31	45	11.7±9.3	2.6±0.4	31.0±0.7	9.4±13.8
	Ours	89	99	2.8±2.6	1.0±0.8	9.8±7.0	61	75	5.2±4.3	0.7±0.1	15.2±2.3	4.5±5.2
Rope B	Base	83	97	3.4±2.8	0.9±0.3	10.4±1.7	33	55	9.5±7.5	1.4±0.7	26.1±2.2	8.3±11.9
	Baseline	83	96	3.4±2.5	0.5±0.3	11.1±2.5	31	53	9.5±7.3	1.0±0.2	25.3±1.7	8.9±14.4
	Ours	95	99	2.3±1.9	1.1±0.7	8.3±3.9	63	83	5.2±4.9	1.0±0.6	20.4±9.0	5.7±9.0
Rope C	Base	65	89	5.2±4.7	1.4±0.1	19.0±3.6	28	53	8.9±5.8	1.6±1.5	22.9±0.3	6.7±9.8
	Baseline	73	91	4.8±3.9	1.7±0.6	18.6±3.1	23	53	9.3±5.9	1.4±0.7	22.4±1.9	7.4±10.5
	Ours	77	96	4.1±3.1	1.5±0.3	15.2±2.9	31	79	6.3±3.3	2.7±0.2	15.7±1.6	5.0±4.2
<i>Mean</i>	Base	72	92	4.5	1.2	14.9	28	50	10.6	1.5	29.2	8.2
	Baseline	74	94	4.2	1.1	13.9	28	51	10.2	1.7	26.2	8.5
	Ours	87	98	3.1	1.2	11.1	52	79	5.6	1.5	17.1	5.1

TABLE II: Hardware striking on three ropes, 25 targets $\times$ 3 swings per cell. Base: nominal simulator; Baseline: simulator identified by a learned network as in Wiggle&Go [6], no correction; Ours: RECAP. SR@ d : success rate (%) within d of the target; 4D goals also require arriving within 10° of the commanded direction at ≥ 1 m/s along it. Distance and Angle (arrival-angle error): mean over all swings, with the std as a subscript. Best and Worst: the lowest and the highest per-target mean distance, with that target’s std. *Mean* is the mean over the three ropes. Best method per rope and goal type in bold.

CEM to search for the parameters $\hat{\eta}$ that best reproduce these trajectories in simulation, minimizing the difference between c and $c_{\hat{\eta}}$, as shown in Fig. 2(b).

Delta policy. For a goal g , policy generation CFM produces the nominal action $\bar{w}$. The delta policy is a conditional flow-matching model with the same architecture that learns to generate real action w_η conditioned on the real goal and system from $\bar{w}$.

$$w_\eta \sim p_\psi(w_\eta | g, \bar{w}, \Delta_{\hat{\eta}}(\bar{w}), \hat{\eta}) \quad (7)$$

where $\Delta_{\hat{\eta}}(\bar{w})$ is the error to the goal at the pass of $\bar{w}$ in the fitted simulator. The delta policy is trained on simulated ropes and rigs with perturbed parameters. Its labels are corrections that CEM finds to improve the pass.

During inference, we generate eight correction actions, replay those, and the nominal action in the fitted simulator, and execute the candidate with the lowest loss J . Thus, each new rope only costs 8 real swings, and every goal afterwards takes a single strike, with no measurement during execution.

D. DeformX 2.0: the GPU co-simulation engine

DeformX [2] couples a Cosserat-rod solver to Isaac Sim for accurate DLO dynamics but is limited by CPU throughput. To support TRACE search, correction-policy training, and candidate verification, we adopt a GPU Cosserat-rod formulation [7] that separates position and rotation updates and solves orientation in closed form, enabling stable integration at large time steps and efficient batched simulation.

We use 100 solver substeps per 60Hz control step with 16 constraint iterations, simulating thousands of rope worlds in parallel with the robot arm in Isaac Sim [24].

To capture aerodynamic effects at swing speeds, we add quadratic cross-flow drag [8], [9], damping each segment’s velocity normal to the local rope tangent at every substep.

DeformX 2.0 generates over 1 million trajectories per hour per GPU. With 8192 parallel environments, its throughput exceeds DeformX by more than $20,000\times$ (Table I).

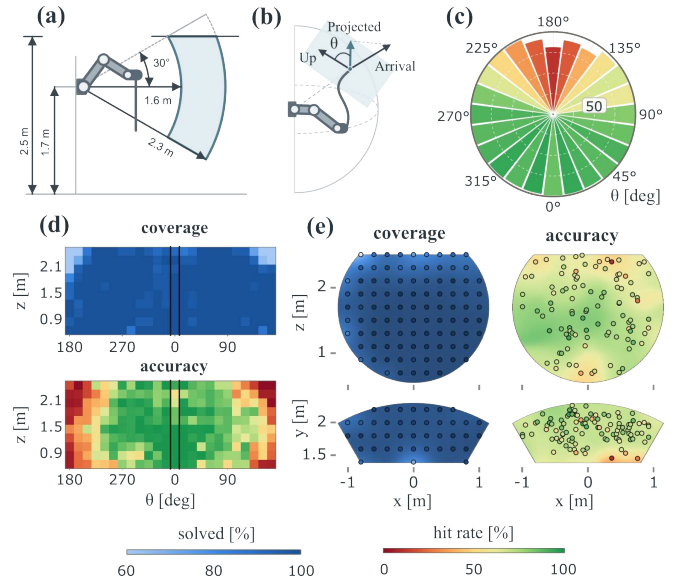


Fig. 3: Experimental setup and simulation results. (a) Targets are selected within the workspace. (b) Arrival angle θ , measured after projecting the rope-tip velocity onto the tangent plane of the arm-centered sphere. (c) Radar-plot radius indicates data-generation coverage for each θ , and color indicates policy hit rate. (d,e) Data-generation coverage and policy accuracy, measured by solved generation targets and hit evaluation targets, respectively.

V. EXPERIMENT

The experiments answer three questions: how fast the simulation is, how accurately the policy strikes a commanded

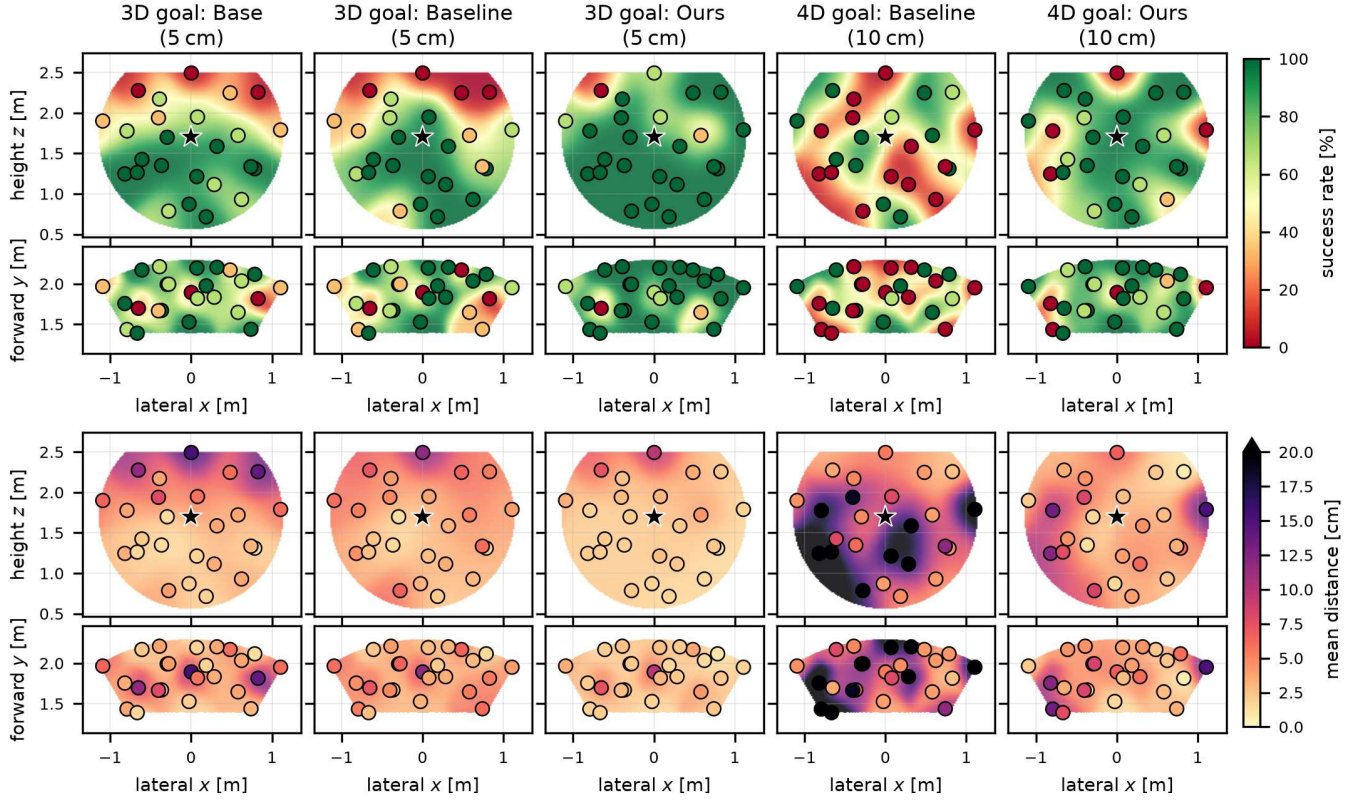


Fig. 4: Workspace maps on Rope A (real world) for 3D goals with Base, Baseline and Ours, and for 4D goals with Baseline and Ours: success rate (upper two rows) and mean tip–target distance (lower two rows) of each target over its three swings. A 3D goal succeeds within 5 cm; a 4D goal, which also requires arriving within 10° of the commanded direction at ≥ 1 m/s, is scored within 10 cm, as the column titles say. The distance scale saturates at 20 cm. Each quantity is shown in the front view (lateral x vs. height z from the wall-mounted base) and, directly below it on the same x axis, in the top view (lateral x vs. forward y). Dots are the evaluated targets; the shaded field is a kernel-weighted interpolation between them, and the black star marks the wall-mounted robot base.

position and arrival direction in simulation and on hardware, and how much the data generation and the rope calibration each contribute. All arms share the setup and protocol below.

A. Experiment Setup

We evaluate the policy in simulation and on a wall-mounted 6-DoF UR5e holding a 0.65 m rigid stick with a 1.0 m rope. The workspace is a 30° cone about the base axis, with radii [1.6, 2.3] m and a maximum height of 2.5 m, as shown in Fig. 3 (a). Joint velocities are limited to 2 rad/s.

The policy uses a 21-parameter action: four evenly timed waypoints for five joints and the swing duration. The final wrist joint is excluded because the stick lies along its rotation axis and have negligible effect on the swing.

Hardware evaluation uses 25 targets with three swings each from separate policy samples. Simulation uses 5700 uniformly sampled workspace targets shared across policies to measure coverage and error.

B. Hit Target Experiment

This experiment evaluates striking on hardware with three ropes that differ in material, diameter and linear density, as shown in Fig. 1. Each rope is tested on the same 25 targets

with two goal types. A 3D goal specifies only the target position. A 4D goal also specifies the arrival direction, drawn at random for each target.

We compare three methods, all of which start from samples of the same policy. Base executes the best candidate in the nominal simulator. Baseline follows Wiggle&Go [6]: a simulation-trained network estimates rope and rig parameters, and the best policy candidate in the identified simulator is executed. Ours is RECAP (Sec. IV-C), fitting the simulator from eight calibration swings recorded once per rope.

A swing succeeds at radius d when the rope tip passes within d of the target, and we report $d = 5$ and 10 cm. A 4D swing must also arrive within 10° of the commanded direction, at no less than 1 m/s along it.

Table II reports the results. RECAP is the strongest method on every rope and both goal types: averaged over the three ropes it raises SR@5 cm from 72% to 87% on 3D goals and from 28% to 52% on 4D goals, against 74% and 28% for Baseline, and it halves the mean 4D miss (10.6 $\rightarrow$ 5.6 cm). The advantage grows with the difficulty of the goal: it is widest on 4D goals, and even on Rope C, RECAP brings most 4D misses inside 10 cm (53% to 79%).

Fig. 4 shows how success rate and miss distance vary

Method	Params (M)	One-shot (%)	Best-of-64 (%)	Miss (cm)	Angle (°)
Ours	11.32	73.5	92.1	3.6	2.4
Naive FM	11.35	67.9	88.9	5.3	3.0
Regression	11.31	21.4	—	24.0	12.4
Nearest-neighbor	—	33.6	—	9.2	6.4

TABLE III: **Learning-method ablation:** We compare our model, naive flow matching, and our network with a regression head at matched capacity, alongside nearest-neighbor lookup in the training data. Miss and angle are means over the one-shot sample on $n = 5700$ held-out goals. Regression falls below the lookup because it averages incompatible motion patterns into invalid swings.

across the Rope A workspace. Both degrade toward the upper edge for Base and Baseline, while RECAP largely removes this degradation, leaving only the top corner weak. Overall, a one-time eight-swing calibration transfers one simulation-trained policy to three different ropes.

VI. ABLATION STUDY

A. Learning distinct motion patterns

We evaluate how the policy architecture represents distinct motion patterns for the same goal. We compare our model with a regression baseline that replaces our flow-matching head with a regression head, a naively conditioned flow-matching model that receives the goal only at its input, and a nearest-neighbor baseline that retrieves the swing associated with the closest training goal. Learned models use matched capacity and training protocols (Table III).

Fig. 5 shows that our model keeps samples compact within each motion pattern while separating distinct patterns. Naive conditioning produces more dispersed samples, while regression can place its prediction between incompatible swings. This distinction matters because an average of successful swings may not succeed.

B. Data generation for policy learning

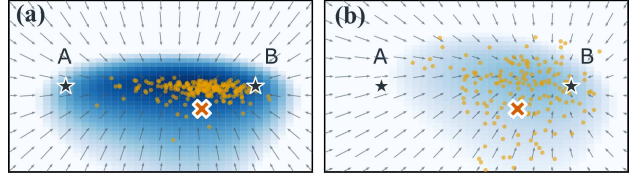
We evaluate the data-generation strategy in terms of workspace coverage, strike accuracy, and trajectory quality. We compare our full TRACE method with CEM initialized from an example swing and our method without an example. We assess both the generated trajectories and the policies trained on the resulting datasets (Table IV).

Our TRACE method achieves broader workspace coverage and higher policy accuracy after candidate selection. The example’s contribution is reflected in lower curvature loss and stronger performance after candidate selection. The example-free variant achieves slightly higher one-shot accuracy, highlighting that trajectory quality and the accuracy of an individual policy sample are distinct properties.

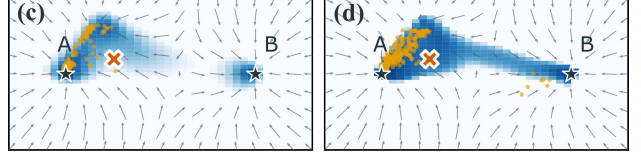
VII. CONCLUSION AND DISCUSSION

This work extends rope-tip targeting to goals that specify both position and arrival direction (4D goals) with a zero-shot

Same motion pattern



Different motion pattern



(e) Concentration

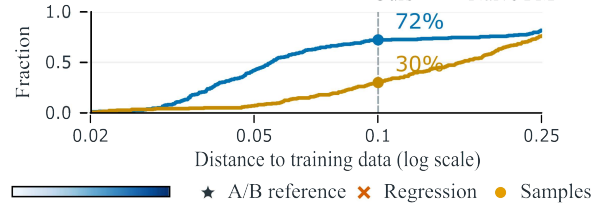


Fig. 5: Learning multiple swings for one goal. We compare goal conditioning in every block (ours, left) with conditioning only at the input (naive FM, right). Stars mark two training swings w_A, w_B associated with goal g ; dots show generated samples. Shading shows a 2D slice of the conditional log-density $\log p(w | g)$, and arrows show the learned flow field at $s = 0.9$. (a,b) For swings from the same motion pattern, our model produces more concentrated samples. (c,d) For swings from different patterns, our model produces more clearly separated density peaks. Regression ($\times$) predicts a single action that can average incompatible swings. (e) For the pair in (c,d), curves show the fraction of samples within a given distance of the nearer training swing, normalized by the pair’s separation in normalized action space.

swing. Our framework learns striking motions in simulation and uses a few calibration swings to adapt them to a real system. The policy achieves 92.1% success within 5 cm and 10° in simulation, while on hardware, across three ropes, calibration raises success within 5 cm to 87% for position goals and 52% for position-and-direction goals.

Our experience suggests a broader insight for deformable object manipulation. We view the dynamics of a deformable object as a complex nonlinear mapping from robot action space to task space outcomes. Manipulation requires solving the inverse map: finding actions that produce a desired outcome. Our observations suggest that this mapping contains locally predictable regions, where small action space changes lead to gradual changes in target space. We observe this structure within motion patterns, while transitions between patterns can be highly sensitive and chaotic. Starting from a swing verified on hardware and extending the search locally produces consistent training data that makes inverse learning easier. These patterns can also benefit sim-to-real

Generator	Coverage (%) $\uparrow$	One-shot (%) $\uparrow$	Best-of-64 (%) $\uparrow$	Mean L_{curv} $\downarrow$
CEM w/ example	49.5	58.5	68.5	0.40
TRACE w/o example	96.8	75.5	83.9	0.51
TRACE	98.3	73.5	92.1	0.33

TABLE IV: Data-generation ablation evaluating workspace coverage, policy accuracy on 5700 random test goals, and trajectory quality measured by mean curvature loss. One-shot evaluates one policy sample; best-of-64 selects the candidate with the lowest TRACE loss. The example seeds higher-quality motion patterns, while TRACE improves coverage and accuracy of the dataset.

transfer by limiting how execution errors and variations in physical properties affect the outcome. Selecting such regions can simplify learning and deployment and may help explain why our policy, trained in simulation, achieves useful hardware performance even before calibration. We believe that identifying similarly robust regions could also benefit other deformable object manipulation tasks.

Improving simulation can reduce reliance on online correction in deformable object manipulation. While feedback compensates for execution errors, a sufficiently accurate simulator can generate better motions in advance. In our framework, its practical value lies in capturing the dynamics well enough to produce effective motions and reliably rank candidates. Improving simulation provides a complementary path to reliable manipulation alongside feedback control.

Several limitations remain. First, our framework targets single-swing rope-tip striking and does not yet address more general forms of dynamic deformable-object manipulation involving sustained contact, obstacle interaction, or multi-stage behavior. Second, execution remains open-loop after a one-time calibration, so disturbances or system changes that occur during a swing cannot be corrected online. Finally, although our calibration reduces the need for an exact physical model, the method still relies on simulation to generate and rank candidate motions, and its robustness may degrade when the real system lies substantially outside the range of dynamics represented during training.

Extending the framework toward closed-loop feedback and broader classes of dynamic DLO interactions is an important direction for future work.

ACKNOWLEDGMENT

The authors used ChatGPT and Claude to assist with language editing and improving the clarity of the manuscript. All technical content, analyses, figures, results, and conclusions were produced and verified by the authors.

REFERENCES

- [1] J. Sanchez, J.-A. Corrales, B.-C. Bouzgarrou, and Y. Mezouar, “Robotic manipulation and sensing of deformable objects in domestic and industrial applications: a survey,” *The International Journal of Robotics Research*, vol. 37, no. 7, pp. 688–716, 2018.
- [2] Y. Yang, X. Fei, L. Wang, C. Li, Z. Dai, H. Kou, L. Li, and H. Choset, “DeformX: A versatile co-simulation framework for deformable linear objects,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2026.
- [3] C. Chi, B. Burchfiel, E. Cousineau, S. Feng, and S. Song, “Iterative residual policy for goal-conditioned dynamic manipulation of deformable objects,” in *Robotics: Science and Systems (RSS)*, 2022.
- [4] H. Zhang, J. Ichnowski, D. Seita, J. Wang, H. Huang, and K. Goldberg, “Robots of the lost arc: Self-supervised learning to dynamically manipulate fixed-endpoint cables,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2021.
- [5] M. C. Nah, A. Krotov, M. Russo, D. Sternad, and N. Hogan, “Manipulating a whip in 3D via dynamic primitives,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 2803–2808.
- [6] A. Jakobsson, A. Mahajan, K. Pullalarevu, K. Suresh, Y. Yao, Y. Mao, B. Duisterhof, S. N. Syed, and J. Ichnowski, “Wiggle and go! System identification for zero-shot dynamic rope manipulation,” arXiv preprint arXiv:2604.22102, 2026, to appear in Conference on Robot Learning (CoRL).
- [7] J. Hsu, T. Wang, K. Wu, and C. Yuksel, “Stable Cosserat rods,” in *ACM SIGGRAPH 2025 Conference Papers*, 2025.
- [8] J. R. Morison, M. P. O’Brien, J. W. Johnson, and S. A. Schaaf, “The force exerted by surface waves on piles,” *Petroleum Transactions, AIME*, vol. 189, pp. 149–154, 1950.
- [9] H. Ekander, N. C. Perkins, and B. W. Richards, “Development of a simulation model for fly casting and application to overhead casting,” *Sports Engineering*, vol. 28, no. 1, p. 2, 2025.
- [10] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [11] W. Peebles and S. Xie, “Scalable diffusion models with transformers,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [12] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, “FiLM: Visual reasoning with a general conditioning layer,” in *AAAI Conference on Artificial Intelligence*, 2018.
- [13] H. Zhu, T. Zhao, X. Ni, J. Wang, K. Fang, L. Righetti, and T. Pang, “Should we learn contact-rich manipulation policies from sampling-based planners?” *IEEE Robotics and Automation Letters*, vol. 10, no. 6, pp. 6248–6255, 2025.
- [14] L. Yang, H. J. T. Suh, T. Zhao, B. P. Graesdal, T. Kelestemur, J. Wang, T. Pang, and R. Tedrake, “Physics-driven data generation for contact-rich manipulation via trajectory optimization,” in *Robotics: Science and Systems (RSS)*, 2025.
- [15] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, and W. Zaremba, “Hindsight experience replay,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [16] D. Ghosh, A. Gupta, A. Reddy, J. Fu, C. Devin, B. Eysenbach, and S. Levine, “Learning to reach goals via iterated supervised learning,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [17] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Robotics: Science and Systems (RSS)*, 2023.
- [18] A. Tong, K. Fatras, N. Malkin, G. Huguet, Y. Zhang, J. Rector-Brooks, G. Wolf, and Y. Bengio, “Improving and generalizing flow-based generative models with minibatch optimal transport,” *Transactions on Machine Learning Research (TMLR)*, 2024.
- [19] C. Chu and H. Xu, “DA-MMP: Learning coordinated and accurate throwing with dynamics-aware motion manifold primitives,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [20] Y. Lee, B. Lee, S. Kim, and F. C. Park, “Motion manifold flow primitives for task-conditioned trajectory generation under complex task-motion dependencies,” *IEEE Robotics and Automation Letters*, vol. 10, no. 7, pp. 7412–7419, 2025.
- [21] S. Kuroki, J. Guo, T. Matsushima, T. Okubo, M. Kobayashi, Y. Ikeda, R. Takanami, P. Yoo, Y. Matsuo, and Y. Iwasawa, “GenDOM: Generalizable one-shot deformable object manipulation with parameter-aware policy,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [22] A. Zeng, S. Song, J. Lee, A. Rodriguez, and T. Funkhouser, “Tossing-Bot: Learning to throw arbitrary objects with residual physics,” *IEEE Transactions on Robotics*, vol. 36, no. 4, pp. 1307–1319, 2020.
- [23] R. Y. Rubinstein, “The cross-entropy method for combinatorial and continuous optimization,” *Methodology and Computing in Applied Probability*, vol. 1, no. 2, pp. 127–190, 1999.
- [24] NVIDIA, “Nvidia isaac sim,” 2026. [Online]. Available: <https://github.com/isaac-sim/IsaacSim>